

Voyage Assistant

Assistant IA conversationnel pour le transport

Rapport de projet — SAE2 BUT3

Ako Christian	IA conversationnelle & RAG, déploiement
Carl Similien	Back-end & intégration (API, base de données)
Noé Cervera	Modèles & évaluation (embeddings, reranker, NER)
Dhanoush Kessavane	Front-end, design & documentation

Encadrement : **Bilal Faye** (LIPN, CNRS UMR 7030)

Promotion 2025–2026

Application en ligne : <https://optimalako-voyage-assistant.hf.space>

Code source : <https://github.com/optmlako2004/chatbot>

Table des matières

1	Introduction et contexte	2
1.1	Objectifs	2
2	Cahier des charges et réponses apportées	2
3	Architecture technique	2
3.1	Vue d'ensemble	3
3.2	Modèle de données	3
4	L'intelligence conversationnelle	3
4.1	LLM — Google Gemini 2.5 Flash	3
4.2	Qu'est-ce que le RAG et la vectorisation ?	3
4.3	La chaîne RAG en deux étapes	3
4.4	Extraction d'entités (NER)	4
4.5	Les trois sources de connaissance	4
4.6	Machine à états pour les actions réelles	4
5	Fonctionnalités principales	4
6	Sécurité	4
7	Déploiement	5
7.1	Choix d'hébergement	5
7.2	Architecture de production	5
7.3	Étapes clés	5
8	Difficultés rencontrées et solutions	5
9	Répartition du travail	6
10	Conclusion et perspectives	6

1 Introduction et contexte

Ce projet s'inscrit dans la **SAE2 du BUT3**, dont l'objectif est de concevoir une application complète intégrant un **assistant conversationnel** fondé sur un modèle de langage (LLM) et une approche **RAG** (*Retrieval-Augmented Generation*), spécialisée sur un domaine métier précis. Nous avons choisi le domaine du **transport de voyageurs** (avion, train, bateau, bus longue distance).

L'application, baptisée **Voyage Assistant**, permet de rechercher et réserver un trajet, de gérer son compte et ses billets, et surtout de **dialoguer avec un assistant IA** qui répond aux questions de voyage et exécute réellement des actions (modification de réservation, réclamation, annulation). Elle est **déployée en ligne** et accessible publiquement, et son interface est **multilingue** (français, anglais, espagnol).

1.1 Objectifs

- Construire une application **full-stack** (frontend, backend, base de données).
- Intégrer un **LLM** et une chaîne **RAG vectorielle** conforme aux concepts vus en cours.
- Garantir des réponses **factuelles** (anti-hallucination) en s'appuyant sur des sources de vérité.
- **Déployer** l'ensemble en ligne, gratuitement.

2 Cahier des charges et réponses apportées

Le tableau suivant met en regard chaque exigence du sujet et sa réalisation concrète.

Exigence	Réalisation
Domaine spécialisé	Transport de voyageurs (avion / train / bateau / bus)
Application full-stack	FastAPI (back) + React (front) + PostgreSQL
LLM	Google Gemini 2.5 Flash
RAG / vectorisation	LangChain + FAISS + embeddings <code>gte-small</code> + reranker CrossEncoder
Base de session	Tables <code>users</code> , <code>chat_sessions</code> , <code>chat_messages</code>
Voix (option)	Web Speech API : dictée vocale + lecture des réponses
Recherche web (bonus)	DuckDuckGo + APIs temps réel (météo, heure, change)
Multilingue (bonus)	Interface et assistant en français, anglais et espagnol
Déploiement en ligne	Hugging Face Spaces + Neon (PostgreSQL)

3 Architecture technique

L'application suit une architecture **client-serveur** classique, enrichie d'une couche d'intelligence artificielle côté serveur.

3.1 Vue d'ensemble

- **Frontend** (`frontend/`) : React 18 (sans étape de build, via Babel navigateur), CSS personnalisé avec thèmes clair/sombre et design responsive, **Web Speech API** pour la voix, et un moteur de **traduction** français / anglais / espagnol.
- **Backend** (`backend/`) : FastAPI, validation Pydantic, ORM SQLAlchemy, et les services d'IA (LLM, RAG, reranker, NER, APIs temps réel).
- **Base de données** : PostgreSQL (Neon) en production, SQLite en développement. Le code est agnostique grâce à une simple variable d'environnement `DATABASE_URL`.

3.2 Modèle de données

Les entités principales sont : `User`, `Route` (catalogue de plus de 182 000 liaisons réelles, tous modes confondus), `Trajet` (occurrence datée d'une route), `Billet`, `Reclamation`, ainsi que `ChatSession` et `ChatMessage` pour la **persistance des conversations**.

4 L'intelligence conversationnelle

Le cœur du projet est l'assistant. Sa philosophie : **un LLM ne suffit pas**. Seul, un modèle de langage *invente* (hallucinations) et ignore les données récentes ou personnelles. Nous l'avons donc couplé à plusieurs sources de vérité via le **RAG**.

4.1 LLM — Google Gemini 2.5 Flash

Le modèle reçoit un *system prompt* strict (rôle, périmètre, refus codifiés), l'historique de la conversation, le contexte récupéré, puis le message courant. Il ne sert qu'à **rédigier** la réponse, jamais à exécuter une action sensible. Une consigne de langue lui est ajoutée pour qu'il réponde dans la langue choisie par l'utilisateur.

4.2 Qu'est-ce que le RAG et la vectorisation ?

RAG (*Retrieval-Augmented Generation*) consiste à **récupérer l'information pertinente avant de générer** la réponse, puis à l'injecter dans le prompt. Pour retrouver cette information par le *sens* et non par mots-clés, on **vectorise** chaque texte : il est converti en un vecteur de nombres (384 dimensions) qui encode sa signification. Deux phrases de sens proche ont des vecteurs proches, *même si elles n'emploient pas les mêmes mots*. C'est ce mécanisme qui permet de relier « je veux annuler » à un article de CGV intitulé « conditions de résiliation ».

4.3 La chaîne RAG en deux étapes

1. **Indexation** : les documents (CGV) sont découpés en *chunks* (`RecursiveCharacterTextSplitter`), vectorisés par le modèle `thenlper/gte-small` (exécuté **en local**), et stockés dans un index **FAISS**.
2. **Recherche** : pour chaque question, un **bi-encoder** remonte rapidement 10 candidats, puis un **cross-encoder** (`ms-marco-MiniLM-L-6-v2`) les **re-trie** finement (*reranking*) pour ne garder que les 3 plus pertinents.

Cette double étape offre la précision du cross-encoder sans payer son coût sur tout le corpus.

4.4 Extraction d'entités (NER)

Pour fluidifier les parcours sensibles, un modèle **NER** (Jean-Baptiste/camembert-ner) extrait le prénom et le nom d'une phrase libre. L'utilisateur peut ainsi tout dire d'un coup au lieu de répondre à plusieurs questions.

4.5 Les trois sources de connaissance

Selon la question, l'assistant interroge la bonne source :

- **RAG documentaire** (FAISS) pour les règles (annulation, bagages...).
- **Base de données** pour les questions personnelles. Exemple : « combien de temps va durer mon vol ? » → lecture du billet de l'utilisateur, **calcul de la durée** à partir des horaires, réponse personnalisée.
- **APIs temps réel et web** pour les données du jour : **Open-Meteo** (météo, heure locale, fuseau), **Frankfurter** (taux de change), **DuckDuckGo** (compléments). Les informations stables (capitale, monnaie d'un pays) ne sont pas appelées : le LLM les connaît déjà.

4.6 Machine à états pour les actions réelles

Modifier, annuler ou réclamer ne sont **jamais** confiés au LLM. Une **machine à états** déterministe pilote le parcours : intention détectée → vérification d'identité (numéro de billet, nom, prénom, date de naissance) → exécution réelle en base → envoi de l'e-mail et du billet PDF mis à jour. Cette séparation **exécution** / **rédaction** élimine les hallucinations d'action.

5 Fonctionnalités principales

- Recherche multimodale (aller simple / aller-retour) avec gestion des escales sur plus de 182 000 liaisons.
- Réservation avec choix de classe et bagages, génération d'un **billet PDF** (ReportLab) et envoi d'un **e-mail de confirmation** (Brevo).
- Authentification **Google** ou e-mail / mot de passe (tokens signés).
- Assistant IA : questions, modification, réclamation, annulation, suivi.
- **Voix** : dictée et lecture des réponses (Web Speech API).
- **Interface multilingue** français / anglais / espagnol, avec sélecteur de langue ; l'assistant répond également dans la langue choisie.
- Persistance et reprise des conversations, notation en fin de session.

6 Sécurité

La sécurité du chatbot repose sur plusieurs couches : nettoyage des entrées, garde-fous contre insultes et **injections de prompt**, anti-extraction d'informations sans identité vérifiée, *system prompt* restrictif, requêtes SQL paramétrées (anti-injection), échappement HTML natif de React (anti-XSS), et triple vérification d'identité. Aucun secret n'est présent dans le code : tout passe par des variables d'environnement.

7 Déploiement

7.1 Choix d'hébergement

La chaîne RAG impose `torch` et deux modèles Transformers en mémoire (environ 700 Mo à 1 Go de RAM). Les offres gratuites de type Render ou Railway (512 Mo) provoquent un dépassement mémoire. Nous avons donc retenu **Hugging Face Spaces** (16 Go de RAM gratuits, conçu pour les modèles ML).

7.2 Architecture de production

Un **unique service** (conteneur Docker) sert à la fois l'API, le RAG et le frontend statique — même origine, donc aucun problème de CORS. La base de données est externe : **Neon** (PostgreSQL serverless), volontairement co-localisée avec le Space (région *us-east-1*) pour minimiser la latence.

7.3 Étapes clés

- **Image Docker** : installation de `torch` en version **CPU** (200 Mo au lieu de 1,2 Go en CUDA), pré-téléchargement des modèles RAG, écoute sur le port 7860.
- **Migration des données** : les 182 511 routes n'existaient que dans le fichier SQLite local ; un script dédié (`migrate_to_postgres.py`) les copie vers Neon.
- **Secrets** configurés côté Hugging Face : `DATABASE_URL`, `AUTH_SECRET`, `GEMINI_API_KEY`, etc.
- **OAuth Google** : ajout de l'URL du Space dans les origines autorisées.

8 Difficultés rencontrées et solutions

Difficulté	Solution
DuckDuckGo limité (202 Ratelimit)	Passage à la librairie <code>ddgs</code> (rotation multi-backend automatique).
Google ne fournit pas la date de naissance	Le formulaire de réservation devient la source de vérité ; vérification tolérante à l'inversion nom/prénom.
Hallucinations sur les actions	Séparation stricte machine à états / LLM.
Recherche d'escaliers lente (15 s)	Index SQL au démarrage + cache en mémoire (→ 0,34 s).
RAG « classique » (retour encadrant)	Réécriture en LangChain + FAISS + embeddings locaux + reranking + NER.
<code>torch</code> CUDA trop lourd (1,2 Go)	Installation de <code>torch</code> CPU dans le Docker.
512 Mo de RAM insuffisants	Choix de Hugging Face Spaces (16 Go).
Migration des 182k routes	Script de copie SQLite → Neon par lots.

9 Répartition du travail

Membre	Contributions
Ako Christian	Conception de l'IA conversationnelle (LLM, chaîne RAG, reranker, NER, APIs temps réel), développement backend et mise en production.
Carl Similien	Développement back-end (API FastAPI, base de données) et travail d'intégration entre les modules.
Noé Cervera	Mise au point et évaluation des modèles (embeddings, reranking, NER), qualité des réponses.
Dhanoush Kessavane	Développement du front-end, conception graphique, tests de l'application et rédaction de la documentation.

10 Conclusion et perspectives

Voyage Assistant répond à l'ensemble des exigences de la SAE2 : application full-stack, LLM couplé à une chaîne RAG vectorielle conforme aux concepts du cours, persistance des sessions, option vocale, interface multilingue, recherche web enrichie d'APIs temps réel, et **déploiement en ligne effectif**. Au-delà de la simple intégration d'un modèle, le projet démontre une architecture où l'IA s'appuie sur des **sources de vérité** pour rester factuelle et utile.

Perspectives : enrichir le RAG avec davantage de documents métier, ajouter des notifications en temps réel, et industrialiser le déploiement (intégration continue, montée en charge).